PART A

Experiment 7

A.1 Aim: Container Orchestration: Deploying and Scaling Applications on Kubernetes

A.2 Prerequisite.

Windows PC, Docker Desktop installed and running
Kubernetes enabled in Docker Desktop (or Minikube installed)
`kubectl` command-line tool installed
Visual Studio Code with YAML extension (recommended

A.3 Outcome

Students will be able to:

- Understand the basics of **Kubernetes and container orchestration**.
- Deploy an application using **Kubernetes Deployment**.
- Create and manage **Pods and Services**.
- **Scale** applications by increasing/decreasing replicas using `kubectl`.
- Monitor and manage the deployed application.

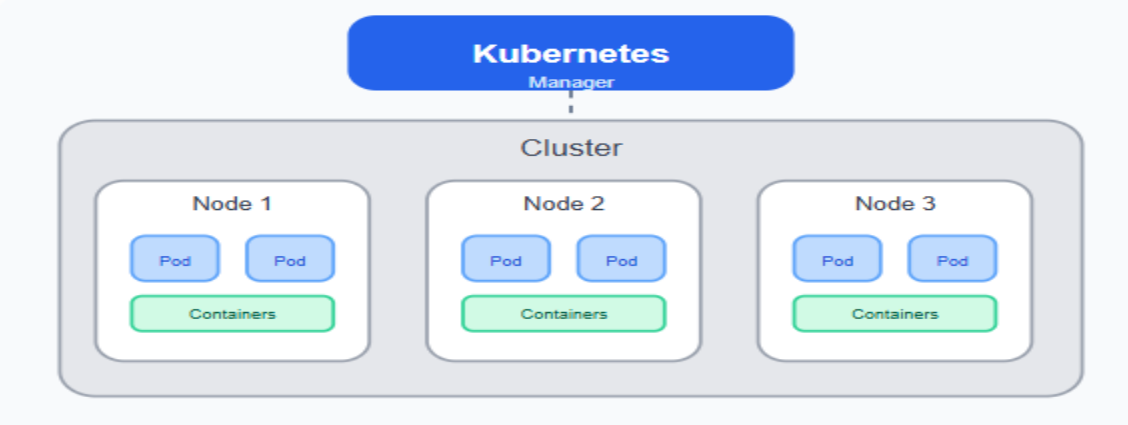
A.4 Theory

What is Kubernetes?

Definition (exam):

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, load balancing, and management of containerized applications.

Think of it like this:



One-line analogy

Important Terminology

Container:A lightweight package containing application + libraries + dependencies.

Pod:Smallest unit in Kubernetes. One or more containers run inside a Pod.

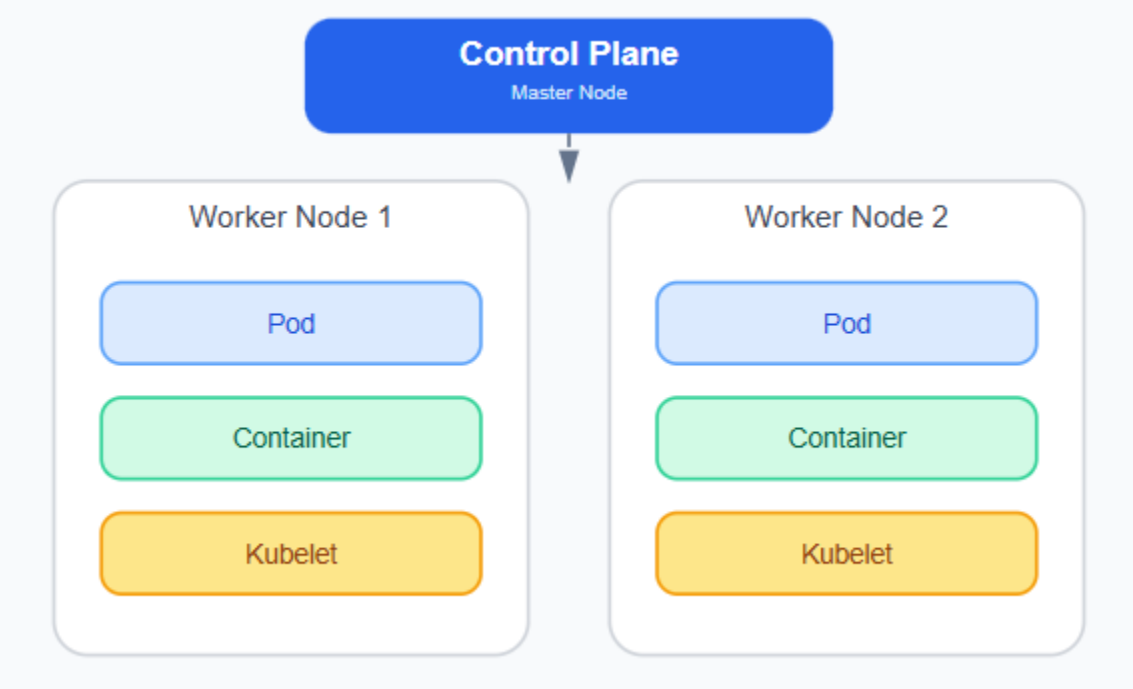
Node:A physical or virtual machine where Pods execute.

Cluster:Collection of multiple Nodes managed by Kubernetes.

Deployment:Defines how many Pods should run and manages updates.

Service:Provides a fixed IP/URL to access Pods.

Architecture



Master Node (Brain):

- Schedules Pods
- Monitors cluster
- Makes decisions

Worker Node:

- Runs containers
- Executes application
- Reports status

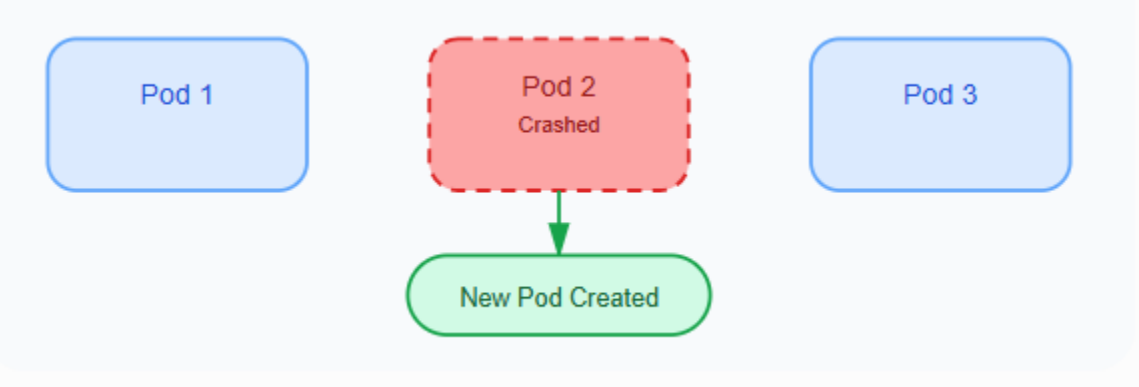
What is Deployment?

Suppose your app needs 3 copies.



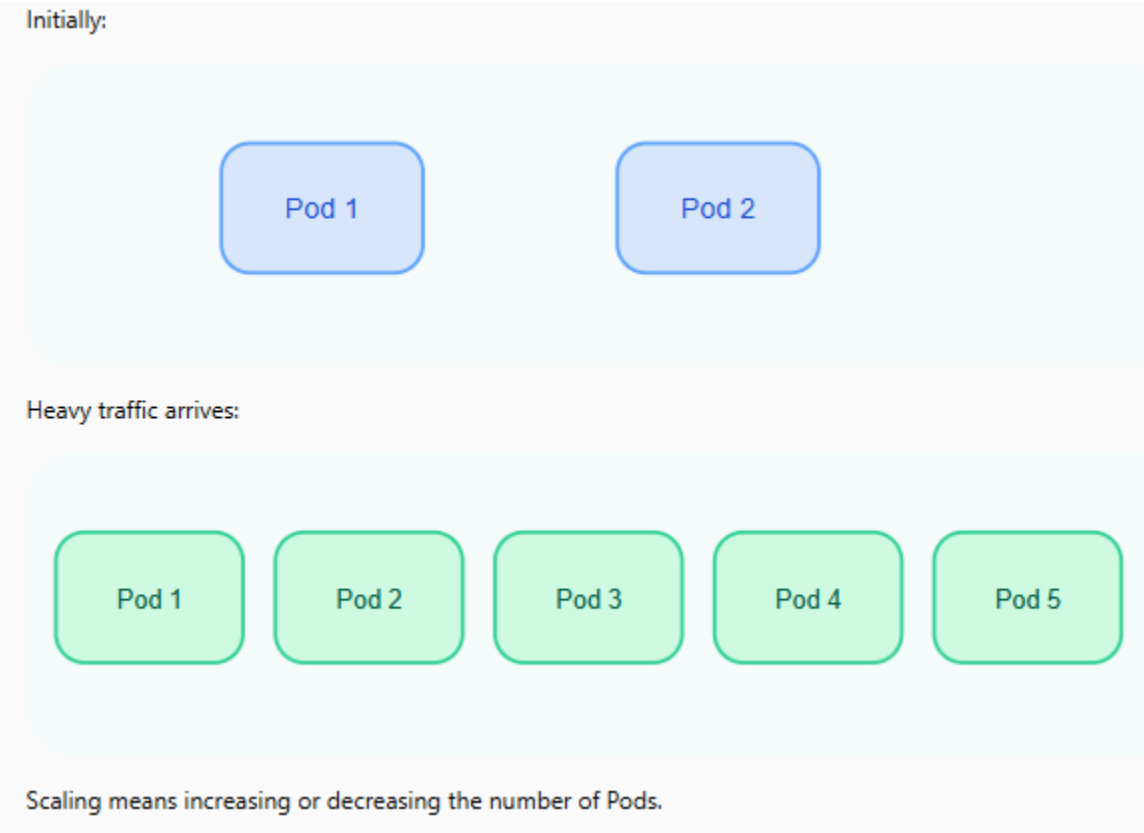
Deployment automatically maintains these 3 Pods.

If one Pod crashes:



Kubernetes detects the failure and creates a new Pod automatically.

What is Scaling?



Command:

kubectl scale deployment myapp --replicas=5

This creates 5 Pods.

Practical Commands (Step-by-Step)

A. Check Cluster

kubectl get nodes

Output:

NAME STATUS

node1 Ready

Meaning: Kubernetes is running.

B. Create Deployment

kubectl create deployment myapp --image=nginx

Creates one Nginx Pod.

C. Check Pods

kubectl get pods

Example:

NAME STATUS

myapp-6d8f7 Running

D. Expose Service

kubectl expose deployment myapp --type=NodePort --port=80

Now the application becomes accessible.

E. Scale Application

kubectl scale deployment myapp --replicas=3

Check again:

kubectl get pods

Output:

myapp-1 Running

myapp-2 Running

myapp-3 Running

YAML File

Instead of typing commands, Kubernetes uses YAML.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
```

```
  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

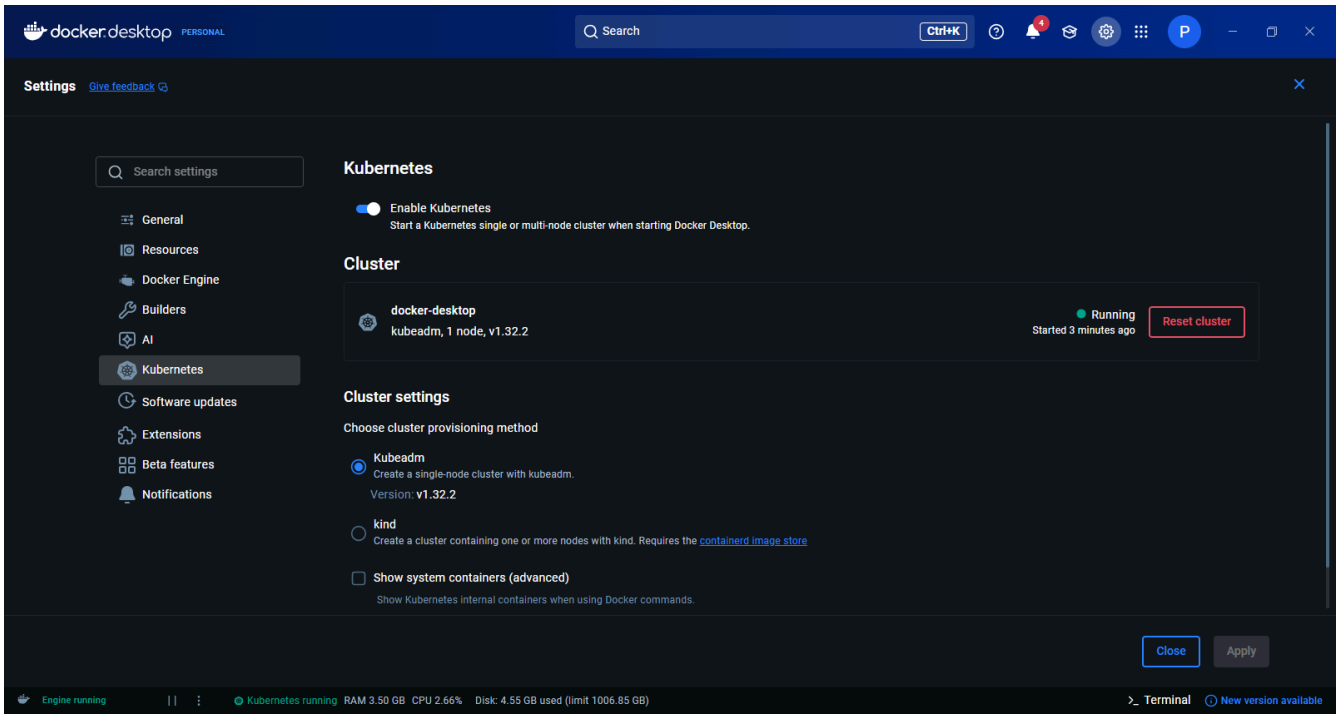
Run it:

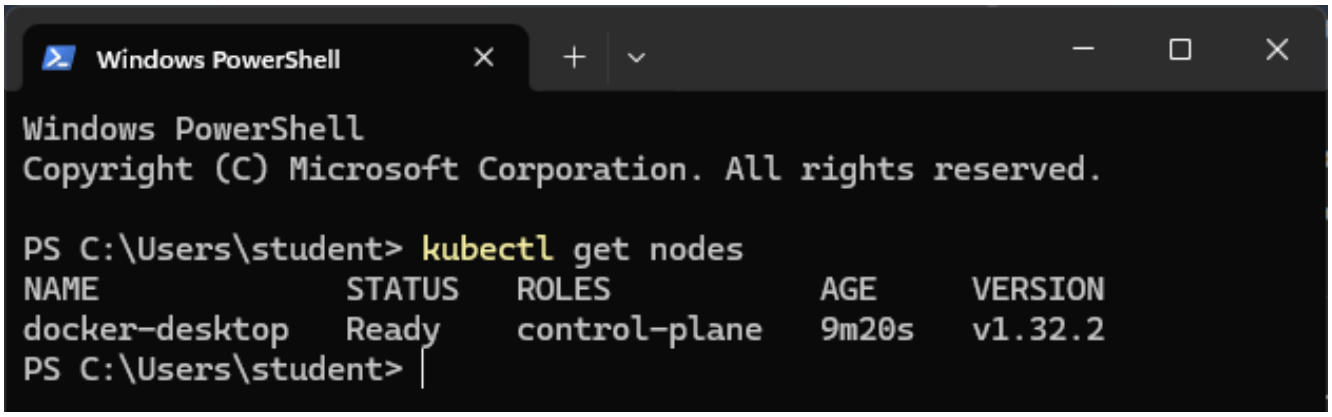
```
kubectl apply -f deployment.yaml
```

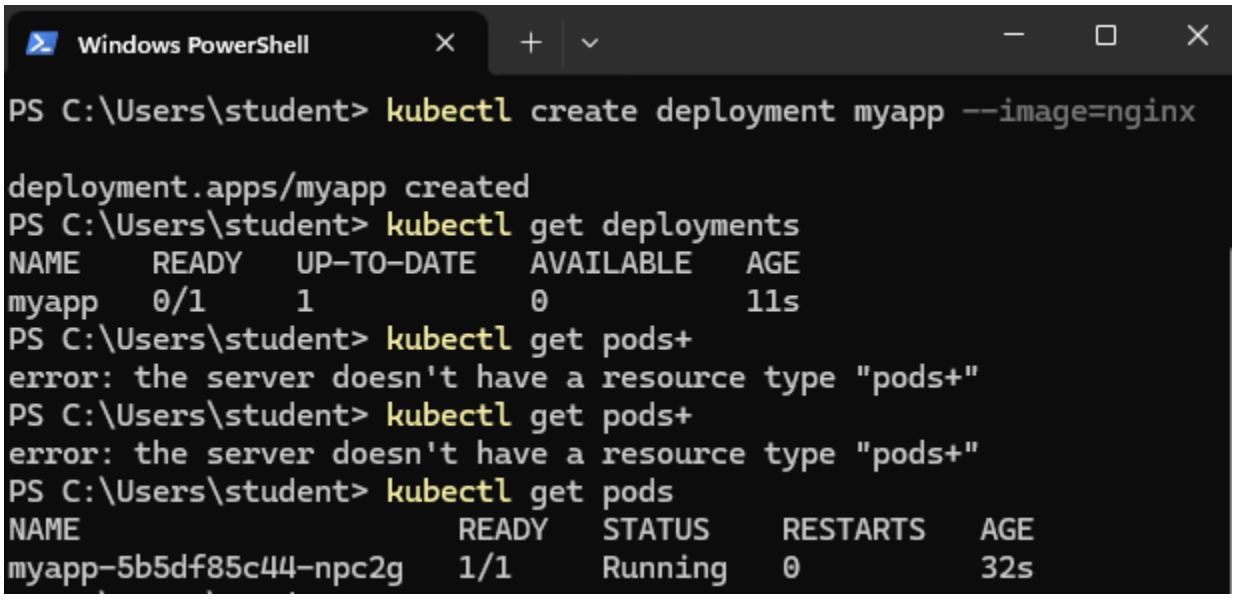
PART B
(PART B: TO BE COMPLETED BY STUDENTS)

Roll No. A27	Name: Prajwal Ashok Zolage
Class: BE–AI&DS	Batch: A1
Date of Experiment: 22/09/2026	Date of Submission: 22/09/2026

B.1 Code Screenshot:

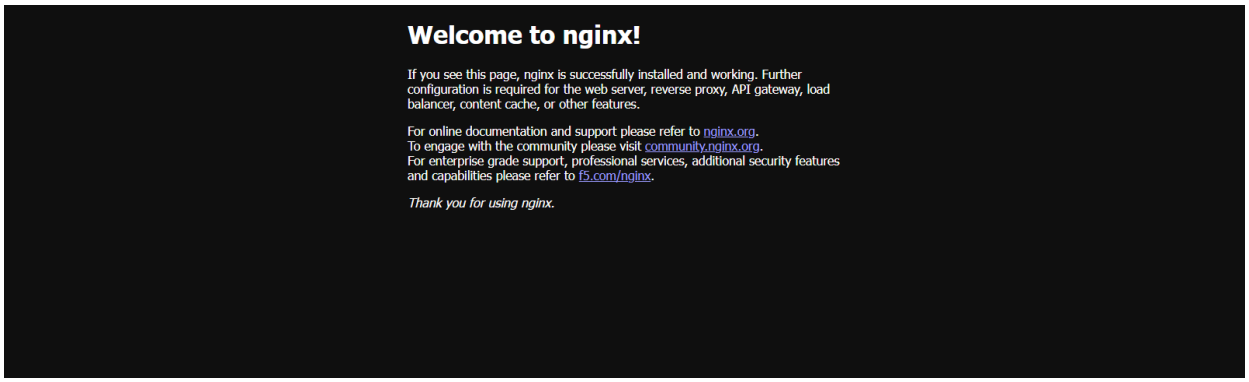


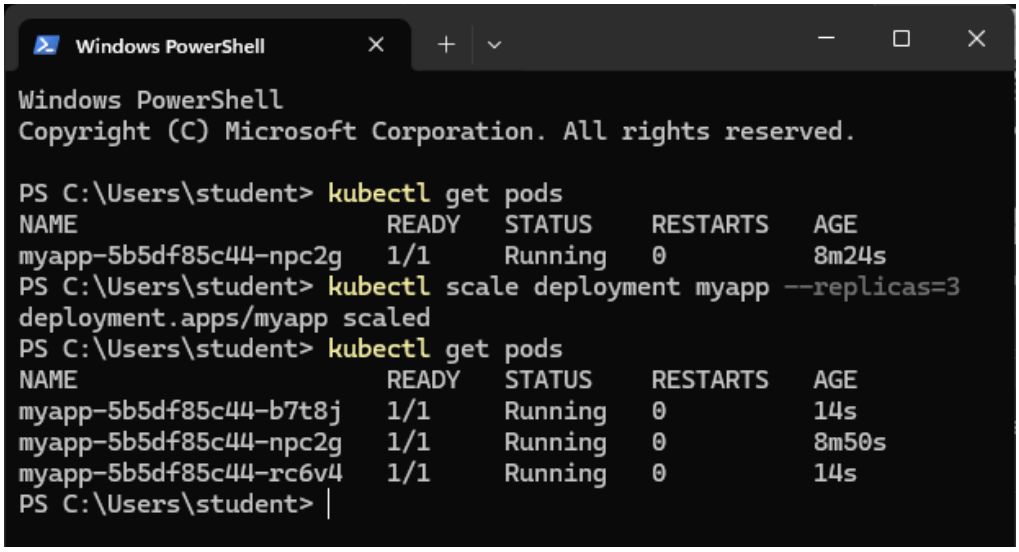




```
PS C:\Users\student> kubectl expose deployment myapp --type=NodePo
rt --port=80
service/myapp exposed
PS C:\Users\student> kubectl get services
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
     AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP
     17m
myapp        NodePort    10.106.241.120   <none>        80:31848/T
CP   9s
PS C:\Users\student>
```

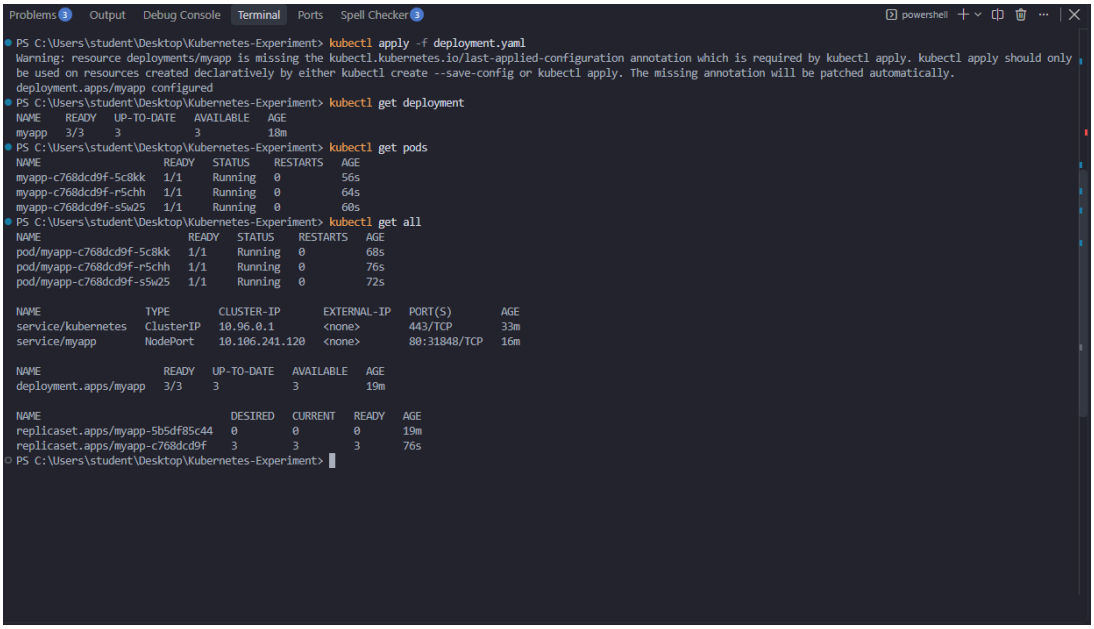
```
PS C:\Users\student> kubectl port-forward service/myapp 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```





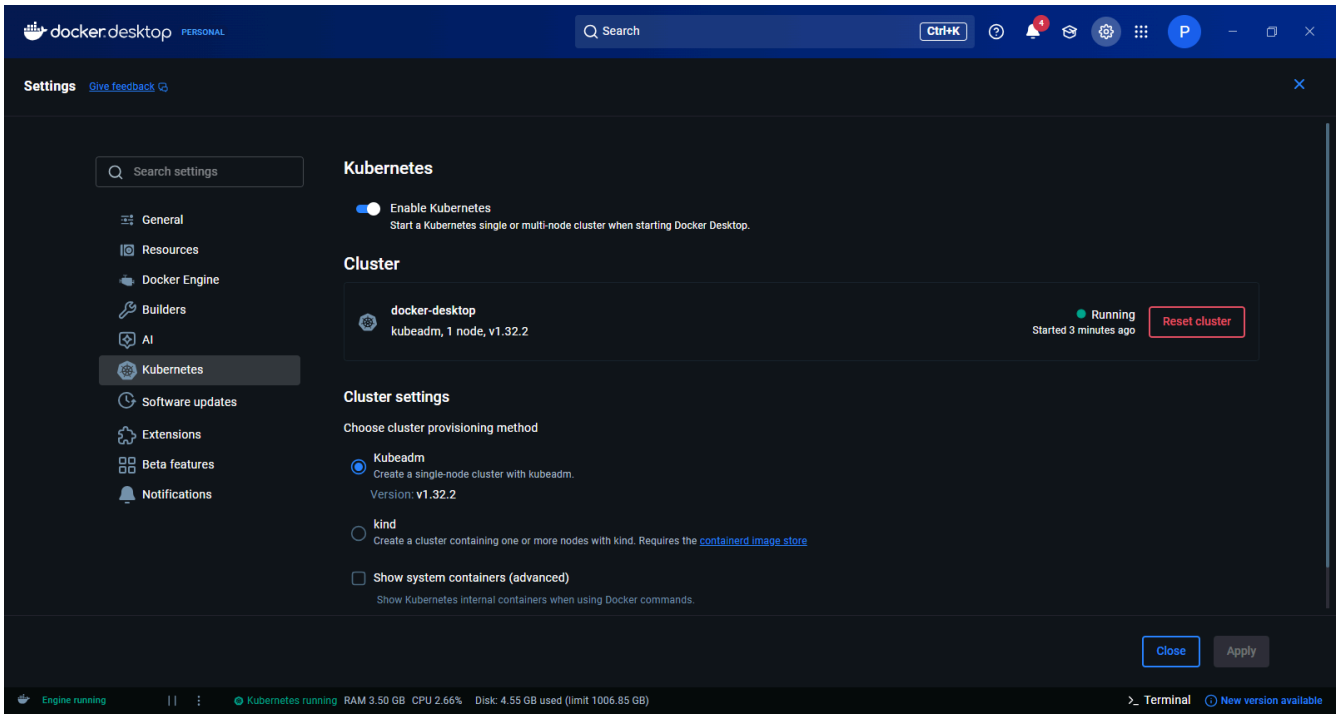

Windows PowerShell
PS C:\Users\student> kubectl get deployment myapp
NAME READY UP-TO-DATE AVAILABLE AGE
myapp 3/3 3 3 10m
PS C:\Users\student> kubectl get all
NAME READY STATUS RESTARTS AGE
pod/myapp-5b5df85c44-b7t8j 1/1 Running 0 2m36s
pod/myapp-5b5df85c44-npc2g 1/1 Running 0 11m
pod/myapp-5b5df85c44-rc6v4 1/1 Running 0 2m36s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 25m
service/myapp NodePort 10.106.241.120 <none> 80:31848/TCP 8m19s
NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/myapp 3/3 3 3 11m
NAME DESIRED CURRENT READY AGE
replicaset.apps/myapp-5b5df85c44 3 3 3 11m
PS C:\Users\student>

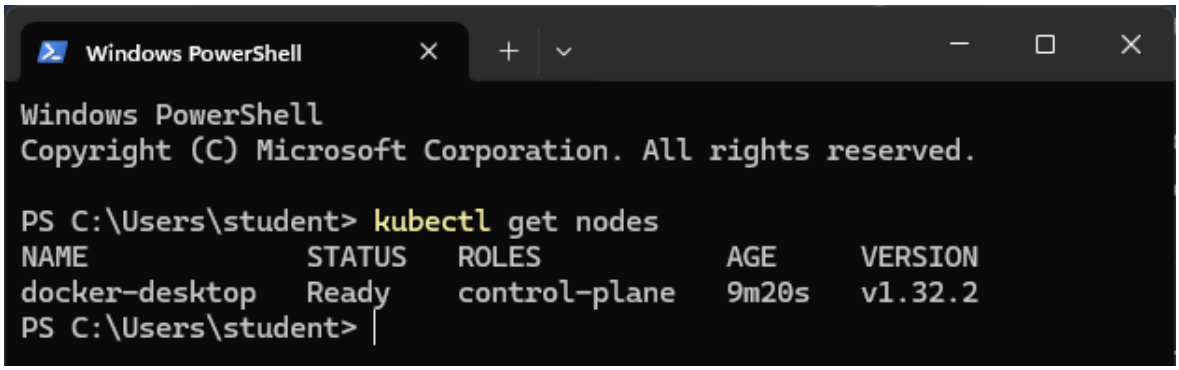

Explorer
Kubernetes-Experiment
deployment.yaml
deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp
spec:
replicas: 3
selector:
matchLabels:
app: myapp
template:
metadata:
labels:
app: myapp
spec:
containers:
- name: nginx
image: nginx
ports:
- containerPort: 80


Problems 3 Output Debug Console Terminal Ports Spell Checker 3
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl apply -f deployment.yaml
Warning: resource deployments/myapp is missing the kubectl.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
deployment.apps/myapp configured
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get deployment
NAME READY UP-TO-DATE AVAILABLE AGE
myapp 3/3 3 3 18m
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get pods
NAME READY STATUS RESTARTS AGE
myapp-c768dcd9f-5c8kk 1/1 Running 0 56s
myapp-c768dcd9f-r5chh 1/1 Running 0 64s
myapp-c768dcd9f-s5w25 1/1 Running 0 60s
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get all
NAME READY STATUS RESTARTS AGE
pod/myapp-c768dcd9f-5c8kk 1/1 Running 0 68s
pod/myapp-c768dcd9f-r5chh 1/1 Running 0 76s
pod/myapp-c768dcd9f-s5w25 1/1 Running 0 72s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 33m
service/myapp NodePort 10.106.241.120 <none> 80:31848/TCP 16m
NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/myapp 3/3 3 3 19m
NAME DESIRED CURRENT READY AGE
replicaset.apps/myapp-5b5df85c44 0 0 0 19m
replicaset.apps/myapp-c768dcd9f 3 3 3 76s
PS C:\Users\student\Desktop\Kubernetes-Experiment>

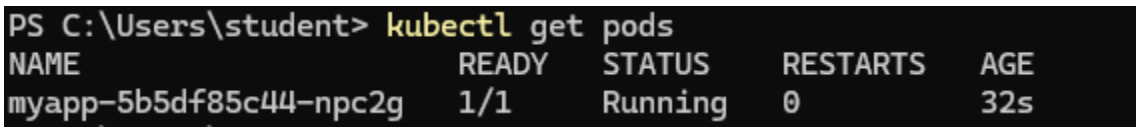
B.2 Input and Output:

1. Kubernetes running

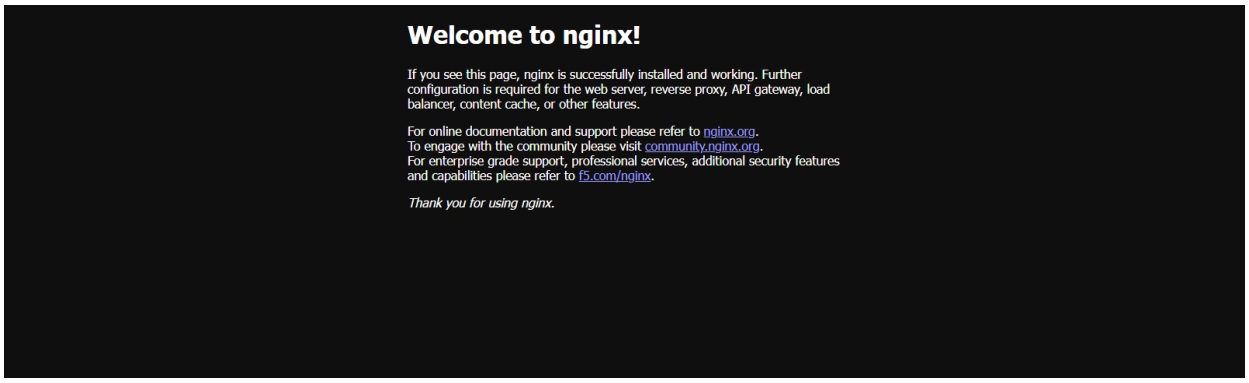




2. Pod creation



3. Nginx website



4. Scaling

```
PS C:\Users\student> kubectl scale deployment myapp --replicas=3
deployment.apps/myapp scaled
PS C:\Users\student> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
myapp-c768dcd9f-5c8kk    1/1     Running   0          14m
myapp-c768dcd9f-r5chh    1/1     Running   0          14m
myapp-c768dcd9f-s5w25    1/1     Running   0          14m
PS C:\Users\student>
```

5. Final Kubernetes status

```
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
myapp   3/3     3            3           18m
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
myapp-c768dcd9f-5c8kk    1/1     Running   0          56s
myapp-c768dcd9f-r5chh    1/1     Running   0          64s
myapp-c768dcd9f-s5w25    1/1     Running   0          60s
PS C:\Users\student\Desktop\Kubernetes-Experiment> kubectl get all
NAME                         READY   STATUS    RESTARTS   AGE
pod/myapp-c768dcd9f-5c8kk    1/1     Running   0          68s
pod/myapp-c768dcd9f-r5chh    1/1     Running   0          76s
pod/myapp-c768dcd9f-s5w25    1/1     Running   0          72s

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
service/kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP        33m
service/myapp        NodePort    10.106.241.120   <none>        80:31848/TCP   16m

NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/myapp   3/3     3            3           19m

NAME                               DESIRED   CURRENT   READY   AGE
replicaset.apps/myapp-5b5df85c44   0         0         0       19m
replicaset.apps/myapp-c768dcd9f    3         3         3       76s
PS C:\Users\student\Desktop\Kubernetes-Experiment>
```

B.3 Conclusion:
The experiment was successfully performed using Kubernetes on Docker Desktop. An Nginx application was deployed using a Kubernetes Deployment and managed using Pods and a NodePort Service. The application was successfully accessed through a web browser. The number of replicas was increased from one to three using the `kubectl scale` command, demonstrating Kubernetes scaling. A YAML configuration file was also created and successfully applied using `kubectl apply`. The experiment provided practical understanding of container orchestration, deployment, service management, scaling, and monitoring using Kubernetes.

Viva Questions & Answers

Question	Answer
What is Kubernetes?	Container orchestration platform.
What is a Pod?	Smallest deployable unit containing containers.
What is Deployment?	Manages Pods and replicas.
What is Scaling?	Increasing/decreasing Pods.
Difference between Docker & Kubernetes?	Docker creates containers; Kubernetes manages many containers.
What is Node?	Machine that runs Pods.

****